

Project 4 — Preference Elicitation for Movie Recommendation

Human-Centric AI · TUHH · Summer semester 2026

This report describes the design of a user study comparing two preference-elicitation interfaces on the IMDB 5000 movie dataset. Task 1 defines a compact feature representation of a movie; Task 2 extends the Bradley-Terry model to full rankings via Plackett-Luce; Task 3 specifies the experimental protocol that would be used to run the study; Task 4 is the Django interface that a participant would use.

Task 1 — Feature representation

A movie is represented by a low-dimensional numerical vector $\mathbf{x} \in \mathbb{R}^d$ so that the user utility $U(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$ is well-defined and every coefficient of $\mathbf{w}$ has an interpretable meaning. The chosen features are:

Group	Feature	Rationale
Quality	imdb_score	Coarse quality signal used by the public.
Physical	duration_z	Length preference (short vs long movies).
Era	year_z	Preference for old vs recent films.
Economics	log_budget_z, log_gross_z	Blockbuster vs indie separation.
Content	14 genre indicators (multi-hot)	The core content-based signal for a movie recommender; each dimension is
Culture	is_english	English vs foreign-language production.
Audience	4 content-rating one-hots (G / PG / PG-13 / R)	Target audience (family vs adult).

The resulting vector has $d \approx 24$ dimensions — small enough that a handful of pairwise comparisons or one or two rankings of ten movies contain useful information about $\mathbf{w}$. We deliberately avoid director/actor identity features (hundreds of near-zero dimensions with sparse support) and text tag features (opaque to the participant).

All numerical features are z-scored so that no single feature dominates the utility because of scale. Missing values are replaced by 0 (i.e. the mean, after standardisation).

Task 2 — Ranking model (Plackett-Luce extension of Bradley-Terry)

The Bradley-Terry model defines the probability that item i is preferred to item j as

$$P(i \succ j) = \exp(U(i)) / (\exp(U(i)) + \exp(U(j))).$$

Design 2 collects full orderings $i_1 \succ i_2 \succ \dots \succ i_n$ of ten movies, so we need to extend the model. The natural extension is the **Plackett-Luce** model:

$$P(i_1 \succ \dots \succ i_n) = \prod_{k=1}^n \exp(U(i_k)) / \sum_{j=k}^n \exp(U(i_j)).$$

Intuition: at each step k , the top-ranked remaining item is selected via a softmax over the utilities of the surviving items. When $n = 2$ the product collapses to a single term and we recover exactly the Bradley-Terry probability — so Plackett-Luce is a genuine extension of it.

Given a set of observed rankings, the parameters $\mathbf{w}$ of the utility function are estimated by **maximum likelihood**. The log-likelihood is the sum, across trials, of the per-step log-softmax terms above; the gradient with respect to $\mathbf{w}$ is

$$\partial \ell / \partial \mathbf{w} = \sum_k [\mathbf{x}_{i_k} - (\sum_{j=k}^n \mathbf{p}_{jk} \cdot \mathbf{x}_j)]$$

where $\mathbf{p}_{jk}$ is the softmax weight of item j at step k . In the code (ml.py) this is implemented as a small gradient-ascent MLE with an isotropic Gaussian prior (L2 regulariser) — sufficient for the online, low-data setting of preference elicitation.

Pairwise responses from Design 1 are trivially rankings of size 2, so the same estimator handles both interfaces — making the two datasets directly comparable in the analysis.

Task 3 — User-study design

Research question

Do participants elicit their preferences more accurately, per unit of elapsed time, using pairwise choices (Design 1) or by ranking a set of ten movies (Design 2)?

Hypotheses

H0. The two interfaces yield preference vectors of equal predictive accuracy on a held-out set of pairwise comparisons.

H1. Design 2 yields higher accuracy per unit elapsed time, because ten-way rankings contain more information per response than a single pairwise choice.

H2. Design 2 has higher subjective load per response than Design 1, so the two interfaces trade off ergonomics against information density.

Design

Between-subjects design with two conditions (Design 1 · pairwise, Design 2 · rank-10). Between-subjects avoids carry-over of learned preferences across interfaces and matches how a real recommender would be deployed (a new user is exposed to one interface at a time).

Assignment to condition is random, balanced by the app (`_assign_condition` in `views.py` picks the currently under-represented condition, breaking ties uniformly at random).

Participants and recruitment

Target sample size: **N = 60** (30 per condition), sufficient for detecting a medium-sized effect (Cohen's $d \approx 0.65$) with power 0.80 at $\alpha = 0.05$ in an unpaired t-test.

Recruitment via TUHH student mailing lists and course noticeboards. Eligibility: adult participants who watch at least one movie per month (self-report). No compensation beyond a debrief at the end. Ethics: this is a low-risk anonymous online study; informed consent is collected on the consent page.

Independent variable

Interface condition (Design 1 vs Design 2).

Dependent variables

Primary. Accuracy of the estimated preference vector $\hat{\mathbf{x}}$ on a held-out set of 20 pairwise comparisons drawn from a separate slice of the IMDB dataset. For each held-out pair (i, j) , the prediction is the item with higher $\hat{\mathbf{x}}_i$; accuracy is the fraction of correct predictions.

Secondary. Mean time per response (ms), reported to the server via `performance.now()`. Time-per-response is a proxy for cognitive load.

Subjective. Optional single-item load rating (1-7 Likert, "How mentally demanding was the task?") at the end of the session.

Procedure

1. Landing page — read description and download report if interested.
2. Consent page — read information, tick consent box, optionally fill age group and prior ML/AI experience.
3. Random condition assignment.
4. Elicitation phase — the assigned interface, repeated for a fixed number of trials (20 pairwise choices, or 5 rankings of ten). Progress bar shown at the top.
5. Debrief — thank-you page showing the participant's condition and response count.

Analysis plan

Fit $\hat{\mathbf{x}}$ per participant using the Plackett-Luce MLE described in Task 2 on the responses collected in that condition. Compute accuracy on the held-out pairs. Compare the two condition means with an independent-samples t-test (Welch's if variances differ), report Cohen's d and a 95% confidence interval

on the difference. If the accuracy distributions are strongly non-normal, fall back to a Mann-Whitney U test. Secondary analysis on time-per-response and subjective load with the same test. Multiple comparisons are corrected with Holm-Bonferroni across the three DVs.

Threats to validity and mitigations

Learning effects. Both conditions have a short practice-free start; the between-subjects design prevents transfer.

Fatigue. Trial counts are set so total time stays well under 10 minutes.

Movie familiarity heterogeneity. Random sampling from the IMDB 5000 yields a mix of well-known and obscure movies; participants may prefer the movies they recognise regardless of features. Reported as a limitation.

Task 4 — Interface implementation

The user-facing interface is a small Django app (project4/) with four screens:

1. **Landing** (/project4/) — brief description, PDF download button, "Start the user study" link.
2. **Consent** (/project4/consent/) — informed-consent text and checkbox, optional demographic categories.
3. **Study** (/project4/study/) — either the pairwise or the ranking screen depending on assigned condition. Progress bar at top, response time captured via `performance.now()`.
4. **Complete** (/project4/complete/) — thank-you page and per-participant response count.

Data is persisted in three Django models: **Participant**, **PairwiseResponse**, and **RankingResponse**. Every response is stored anonymously (only a random `participant_id` UUID, no personal identifier). The researcher inspects the data via `/admin`.

The ranking UI uses up/down buttons on each row (vanilla JavaScript, no external library) so it works with keyboard and on mobile. Item numbering re-renders automatically as items move.

Code is split into `ml.py` (data + features + Plackett-Luce), `models.py`, `forms.py`, `urls.py`, `views.py` (thin controllers), `admin.py` and `tests.py` (pure-logic tests).

Reproducing this document

This report is generated by `python manage.py build_report_p4`, which uses ReportLab. The management command lives in `project4/management/commands/build_report.py`.